



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD: INFORMÁTICA Y ELECTRÓNICA
ESCUELA DE INGENIERÍA EN SISTEMAS
CARRERA: SOFTWARE

GUÍA DE LABORATORIO DE APLICACIONES INFORMÁTICAS II

PRÁCTICA No. 3

1. DATOS GENERALES:

NOMBRE DEL ESTUDIANTE

Andres Romero

CODIGO DEL ESTUDIANTE

7274

FECHA DE REALIZACIÓN:

2026/07/08

FECHA DE ENTREGA:

2025/07/08

2. OBJETIVO(S):

2.1. GENERAL

Solucionar los problemas hallados en el sistema de hospitalaria analizada mediante el mantenimiento de su código fuente para mejorar el funcionamiento del sistema

2.2. ESPECÍFICOS

- Registrar los módulos o áreas del sistema donde se realizará el mantenimiento.
- Analizar los problemas que se resolverán en el sistema a fin de priorizar cada uno.
- Realizar pruebas del funcionamiento del sistema para comprobar la resolución de problemas y cumplimiento de criterios de aceptación.

3. METODOLOGÍA

La presente practica se desarrolló mediante una metodología aplicativa, orientada al analisis, ejecución y comprobación del mantenimiento realizado al sistema hospitalario

seleccionado y analizado en prácticas anteriores, empleando metodologías y marcos de trabajo centrados en el mantenimiento de un producto software. Este enfoque permitió un desarrollo estructurado y organizado de los diferentes procesos en el mantenimiento desde la comprensión del software hasta su posterior ejecución.

4. EQUIPOS Y MATERIALES:

- Dell Inspiron 5593
- Visual Studio Code
- Elearning ESPOCH
- Acceso a internet
- Bibliografía

5. MARCO TEORICO:

Mantenimiento de Software

El mantenimiento de software es un proceso integral que ocurre después de la entrega del producto y busca corregir defectos, mejorar el rendimiento u otros atributos, o adaptar el producto a un entorno modificado. En el contexto de un sistema hospitalario, este proceso se divide principalmente en cuatro tipos:

- **Mantenimiento Correctivo:** Orientado a resolver fallos que impiden el funcionamiento normal del sistema.
- **Mantenimiento Adaptativo:** Se realiza para que el software siga siendo utilizable en un entorno cambiante (por ejemplo, nuevos prefijos telefónicos).
- **Mantenimiento Perfectivo:** Busca añadir nuevas funcionalidades o mejorar las existentes para optimizar la experiencia del usuario (como asegurar la autenticidad del correo electrónico).
- **Mantenimiento Preventivo:** Enfocado en prevenir fallos futuros y mejorar la mantenibilidad (como el control de sesiones en el dashboard).

Ingeniería Inversa como Herramienta de Diagnóstico

Antes de ejecutar cualquier cambio en el código fuente, es esencial comprender la arquitectura existente. La ingeniería inversa es el proceso de analizar un sistema para identificar sus componentes y relaciones, permitiendo representarlo en un alto nivel de abstracción. Esta técnica es fundamental para el mantenimiento cuando no se cuenta con documentación actualizada, ya que permite:

- Reconstruir la lógica del negocio y las dependencias del sistema.
- Realizar un análisis estático y dinámico para derivar diagramas de clases y componentes.
- Facilitar un diagnóstico claro de la capacidad operativa del software antes de su intervención.

Metodologia ITIL

Para una gestión estructurada del mantenimiento, se aplican las buenas prácticas de ITIL, que permiten alinear los servicios de TI con las necesidades del sistema. Dentro de este marco, destacan:

- **Gestión de Problemas:** Se encarga de analizar las causas raíz de incidencias repetitivas para encontrar soluciones permanentes.
- **Diseño del Servicio:** Se refiere al diseño de los procesos y la tecnología necesarios para resolver los problemas encontrados.
- **Transición del Servicio:** Se centra en implementar cambios en el código mientras se mantiene la estabilidad del estado actual del servicio.

Marco de Trabajo MoSCoW para la Priorización

Dado que los recursos y el tiempo en el mantenimiento son limitados, se utiliza el método MoSCoW para priorizar los requerimientos de reparación y mejora. Este acrónimo clasifica las tareas en:

- **Must have (Debe):** Requisitos vitales para el éxito del sistema (ej. control de ingreso al dashboard).
- **Should have (Debería):** Requisitos críticos pero que admiten soluciones alternativas temporales.
- **Could have (Podría):** Requisitos deseados que se implementan si el tiempo lo permite.
- **Won't have (No tendrá):** Requisitos que no se incluirán en la versión actual pero se consideran para el futuro.

6. PROCEDIMIENTO:

1. Registro de módulos / áreas a intervenir:

- **dashboard/dashboard.py:** control de acceso al panel administrativo, persistencia de sesión, cierre de sesión, auditoría de eventos, reservación rápida en el sidebar, validación de prefijos telefónicos, validación de número local y validación de correo electrónico en los formularios de huésped.
- **dashboard/dashboard.py:** reubicación de los CRUD administrativos de rooms, room_types, guests y reservations hacia sus respectivas tarjetas principales del dashboard.
- **frontend/index.html, frontend/script.js y frontend/style.css:** formulario de reservación pública con prefijo telefónico, composición del número final y validación visual consistente con el sistema.
- **backend/main.py:** validación de reservas, normalización de datos de contacto y endurecimiento de reglas de validación para evitar registros inconsistentes.
- **database/resort_db.sql:** esquema de almacenamiento de reservas y huéspedes, revisado para confirmar que los cambios no alteran la estructura existente.

2. Análisis y priorización:

Requisito 1: Controlar el ingreso al dashboard administrador

- **Severidad:** alta.

- **Impacto:** Protege el área administrativa del sistema.
- **Urgencia:** El acceso debe quedar restringido antes de realizar operaciones críticas.

Requisito 2: Mantener la sesión iniciada hasta que el usuario cierre sesión

- **Severidad:** Alta.
- **Impacto:** Evita interrupciones innecesarias en el trabajo administrativo.
- **Urgencia:** Parte del flujo diario de uso del panel.

Requisito 3: Validar el número telefónico del cliente

- **Severidad:** Alta.
- **Impacto:** El teléfono es un dato operativo clave para la reserva y el contacto con el huésped.
- **Urgencia:** Evita almacenar información incompleta o inválida.

Requisito 4: Validar el correo electrónico del cliente

- **Severidad:** Alta.
- **Impacto:** El correo es un canal principal de comunicación.
- **Urgencia:** Reduce errores en los datos de contacto.

Requisito 5: Permitir la selección de prefijos telefónicos de país

- **Severidad:** Media-alta.
- **Impacto:** Mejora la normalización del número y la usabilidad del formulario de reservación.
- **Urgencia:** La captura de datos depende de este componente.

Requisito 6: Reposicionar los CRUD administrativos a sus tarjetas correspondientes

- **Severidad:** Media-alta.
- **Impacto:** Mejora la organización operativa del dashboard sin afectar la base de datos.
- **Urgencia:** La navegación del panel debe coincidir con la estructura funcional.

3. Implementación de cambios:

Buenas prácticas aplicadas

- Se mantuvo la lógica funcional existente y se reforzaron únicamente los puntos de validación y organización necesarios.
- Se centralizaron validaciones reutilizables para teléfono y correo, evitando duplicar lógica en múltiples formularios.

- Se conservó el estilo visual del sistema para no romper la experiencia de uso del dashboard ni del frontend.
- Se preservó la compatibilidad con el despliegue en contenedores y con la estructura actual del proyecto.

Estándares de código y consistencia

- Se usaron nombres descriptivos para funciones, variables y estados de sesión.
- Se aplicó validación previa al guardado para reducir errores de datos y evitar correcciones posteriores.
- Se mantuvo el enfoque modular del dashboard, separando autenticación, reservación rápida, CRUD y archivado en bloques independientes.
- Se reorganizó la interfaz administrativa sin alterar la lógica de persistencia ni la estructura de la base de datos.

Cambios funcionales definidos

- Se añadió control de autenticación con persistencia de sesión y cierre manual en el dashboard.
- Se incorporó validación del teléfono local con un máximo de 10 dígitos en la reservación rápida.
- Se agregó un selector de prefijo de país en los formularios de agregar y editar huésped.
- Se reforzó la validación del correo electrónico en los formularios de huésped antes de guardar.
- Se reubicaron los CRUD de rooms, room_types, guests y reservations en sus tarjetas principales del dashboard.
- Se mantuvo el flujo de captura de prefijos telefónicos en la reservación pública y en la reservación rápida del dashboard.

4. Pruebas:

Inicio de sesión válido.

Se ingresó una contraseña correcta en el formulario del dashboard y se confirmó que el sistema permitiera el acceso sin mostrar errores. Luego se verificó que la interfaz principal cargara normalmente.

Rechazo de credenciales inválidas.

Se probó con una contraseña incorrecta para validar que el sistema bloqueara el ingreso. También se comprobó que apareciera el mensaje de error correspondiente sin abrir el panel.

Persistencia de sesión.

Después de iniciar sesión, se recargó la página del navegador para verificar que la autenticación siguiera activa. Se confirmó que el dashboard permaneciera abierto sin pedir inicio de sesión nuevamente.

Cierre de sesión manual.

Se utilizó el botón de cerrar sesión del dashboard para validar que la sesión se eliminara correctamente. Después se comprobó que el sistema regresara a la pantalla de acceso.

Reservación rápida válida.

Se seleccionó un prefijo de país, se ingresó un número local válido y se completaron los demás campos obligatorios. Al ejecutar la reserva, se verificó que el registro se guardara sin errores.

Teléfonos con más de 10 dígitos.

Se intentó ingresar un número local más largo que el permitido para confirmar que el sistema lo rechazara. La validación evitó que se habilitara el guardado o que la reserva continuara.

Alta y edición de huéspedes válidas.

Se probó crear y modificar un huésped usando correo con formato correcto y un teléfono local válido con su prefijo. El sistema aceptó los datos y actualizó el registro sin inconvenientes.

Correos inválidos.

Se ingresaron direcciones con formato incorrecto para validar que el sistema no permitiera guardar el formulario. La prueba confirmó que el mensaje de validación aparecía antes del registro.

CRUD administrativos en sus tarjetas.

Se navegó por las tarjetas de Rooms, Reservations y Guest Info para comprobar que cada CRUD estuviera en su lugar. También se verificó que crear, editar y eliminar siguiera funcionando dentro de cada sección.

Criterios de aceptación

- El dashboard solo debe abrirse con credenciales correctas.
- La sesión debe mantenerse activa hasta que el usuario decida cerrarla.
- El sistema debe rechazar números telefónicos locales que excedan el límite definido.
- El sistema debe rechazar correos electrónicos con formato inválido.
- El formulario debe permitir seleccionar un prefijo de país para componer correctamente el número telefónico.
- Los CRUD administrativos deben estar disponibles en las tarjetas correspondientes y conservar su funcionamiento normal.

5. Despliegue:

Procedimiento de despliegue del sistema modificado

1. Ubicarse en la carpeta del proyecto Hotel-Management-System-maintained.
2. Reconstruir y levantar los contenedores con Docker Compose.
3. Verificar que db, backend, dashboard y frontend se encuentren operativos.
4. Acceder al dashboard administrativo y confirmar:
 - autenticación correcta,
 - persistencia de sesión,
 - cierre manual de sesión,
 - validación de teléfono y correo en huéspedes,
 - reubicación de los CRUD en sus tarjetas.
5. Probar la reservación pública y la reservación rápida para verificar que el flujo de datos se mantiene estable.

Consideración de despliegue

- Los cambios se diseñaron para no alterar la lógica de reservas, disponibilidad de habitaciones ni persistencia de datos.
- El mantenimiento se centró en seguridad de acceso, validación de datos y reordenamiento de la interfaz administrativa.

7. CONCLUSIONES Y RECOMENDACIONES:

Conclusiones

- Se lograron registrar e intervenir con éxito los módulos críticos del sistema, específicamente en dashboard/dashboard.py, frontend/index.html y backend/main.py, permitiendo una ejecución del mantenimiento enfocada en la seguridad del acceso, el reordenamiento de la interfaz y la validación de datos de contacto.
- El análisis y la priorización de los problemas mediante el método MoSCoW permitieron determinar que requerimientos como el control de ingreso al dashboard y la persistencia de sesión tenían una alta severidad y urgencia, lo que facilitó un desarrollo estructurado y eficiente de las soluciones.
- Las pruebas de funcionamiento aplicadas, que validaron desde el rechazo de credenciales inválidas hasta la correcta captura de prefijos telefónicos, confirmaron que los problemas fueron resueltos satisfactoriamente y que el sistema cumple con todos los criterios de aceptación definidos.

Recomendaciones

- Se recomienda mantener el enfoque modular y la consistencia visual empleados en esta intervención para futuros mantenimientos, asegurando que cualquier cambio en los módulos de autenticación o formularios no altere la lógica de persistencia ni la estructura de la base de datos.
- Es aconsejable aplicar formalmente el marco de trabajo MoSCoW e ITIL en los próximos ciclos de vida del software para identificar con antelación qué componentes críticos requieren una actualización o migración tecnológica antes de que queden obsoletos.
- Se sugiere integrar los protocolos de prueba utilizados en esta práctica (casos válidos y fallidos) dentro de una base de conocimiento técnica oficial, garantizando que el personal futuro pueda realizar pruebas de regresión seguras y mantener la estabilidad operativa del sistema hospitalario.